

Guia para entender y defender el programa

Material de estudio para comprender el código y defender el trabajo práctico oralmente.

1. Objetivo general

El proyecto implementa una aplicación de escritorio en Java Swing para simular una carrera de caballos.

La consigna pide que un jugador pueda:

- ingresar su nombre y mail;
- seleccionar un caballo disponible;
- iniciar una carrera;
- ver el avance de los caballos en pantalla;
- conocer el ganador;
- sumar puntaje según el resultado.

Además, el proyecto aplica conceptos vistos en clase: programación orientada a objetos, MVC, DTO, Singleton, DAO/JDBC, GRASP y SOLID.

Una forma simple de explicarlo oralmente es:

El sistema separa la interfaz gráfica, la lógica de negocio y la persistencia. La vista solo muestra datos y recibe acciones del usuario. El controlador coordina esas acciones. El servicio aplica las reglas del juego. El modelo representa jugadores, caballos y carreras. La base de datos se accede mediante DAOs.

2. Como se usa la aplicación

El flujo normal de uso es:

1. Se abre la ventana principal.
2. La aplicación consulta los caballos disponibles en MySQL.
3. El usuario ingresa nombre y mail.
4. El usuario selecciona un caballo.
5. Al crear el jugador, se guarda en la base de datos.
6. El usuario inicia la carrera.
7. Un Timer de Swing hace avanzar la carrera cada pocos milisegundos.
8. La pista se repinta y muestra los caballos avanzando.
9. Cuando un caballo llega a la meta, se calcula el ganador.

10. Se calcula el puntaje del jugador.

11. Se actualiza el puntaje acumulado y se guarda el resultado de la carrera.

3. Arquitectura por capas

El programa esta dividido en paquetes para que cada parte tenga una responsabilidad clara.

views

Contiene la interfaz grafica Swing.

Clase principal:

- `MainView`

Responsabilidades:

- mostrar el formulario del jugador;
- mostrar el combo de caballos;
- mostrar la pista;
- escuchar botones;
- usar un `Timer` para animar la carrera;
- mostrar mensajes al usuario.

La vista no deberia calcular el ganador ni guardar datos en la base. Eso mantiene bajo el acoplamiento.

controllers

Contiene el controlador.

Clase principal:

- `JuegoController`

Responsabilidades:

- recibir las acciones de la vista;
- validar datos simples relacionados con la entrada;
- llamar al servicio correspondiente;
- devolver DTOs a la vista.

Es la puerta de entrada entre Swing y el negocio.

services

Contiene la logica de negocio y la coordinacion de casos de uso.

Clases principales:

- JuegoService
- PuntajeService

Responsabilidades de JuegoService:

- cargar caballos;
- crear jugadores;
- iniciar carreras;
- avanzar la carrera;
- actualizar puntaje;
- guardar resultados.

Responsabilidad de PuntajeService:

- calcular los puntos segun la posicion del jugador.

Separar el puntaje en su propia clase ayuda a cumplir el principio de responsabilidad unica.

model

Contiene las clases del dominio.

Clases principales:

- Jugador
- Caballo
- Carrera
- Pista
- ResultadoCaballo

Estas clases representan los conceptos reales del problema. Aca esta la parte mas importante de la orientacion a objetos.

model.avance

Contiene las distintas estrategias de avance de los caballos.

Clases principales:

- EstrategiaAvance
- AvanceVelocista
- AvanceResistente
- AvanceEquilibrado

- EstrategiaAvanceFactory

Esto permite que cada tipo de caballo avance de una forma distinta sin llenar la clase Caballo de muchos if.

dto

Contiene objetos de transferencia de datos.

Clases principales:

- CaballoDTO
- JugadorDTO
- CarreraEstadoDTO

Un DTO no tiene logica de negocio. Solo transporta datos entre capas, especialmente entre controlador y vista.

repository y repository.mysql

Contienen el acceso a datos.

Interfaces:

- CaballoRepository
- JugadorRepository
- CarreraRepository

Implementaciones MySQL:

- MySqlCaballoRepository
- MySqlJugadorRepository
- MySqlCarreraRepository

La idea es que el servicio no tenga SQL adentro. El servicio sabe que necesita guardar o consultar, pero no sabe como se arma el INSERT o el SELECT.

persistence

Contiene la conexion a la base.

Clase principal:

- DatabaseConnection

Esta clase aplica Singleton para mantener centralizada la conexion JDBC.

4. Modelo del dominio

Jugador

Representa a la persona que juega.

Atributos principales:

- id;
- nombre;
- email;
- puntaje acumulado;
- caballo seleccionado.

Metodos importantes:

- seleccionarCaballo(Caballo caballo);
- sumarPuntos(int puntos).

Explicacion oral:

El jugador conoce su puntaje y el caballo que elige. Cuando termina una carrera, se le suman puntos segun la posicion obtenida.

Caballo

Representa a cada competidor.

Atributos principales:

- id;
- nombre;
- velocidad base;
- resistencia;
- energia actual;
- distancia recorrida;
- factor de rendimiento;
- estrategia de avance.

Metodos importantes:

- avanzar(double distanciaTotal, Random random);
- prepararParaCarrera(Random random);
- reiniciar().

Explicacion oral:

El caballo tiene atributos que influyen en su rendimiento. No todos avanzan igual porque cada caballo tiene una estrategia de avance. Tambien tiene energia, que baja durante la carrera.

Carrera

Representa la competencia.

Atributos principales:

- jugador;
- pista;
- lista de caballos;
- estado de finalizacion.

Metodos importantes:

- `avanzar()`;
- `obtenerRanking()`;
- `obtenerGanador()`;
- `obtenerPosicionJugador()`.

Explicacion oral:

Carrera es experta en determinar el ganador porque conoce la pista, los caballos y sus distancias recorridas. Por eso aplica GRASP Information Expert.

Pista

Representa la distancia total de la carrera.

En este proyecto se usa una distancia de 500 metros desde el controlador.

5. Como funciona el avance de los caballos

Cada caballo avanza en pequenos pasos. La carrera no se calcula toda de una vez, sino que se actualiza repetidamente.

El flujo es:

1. El Timer de Swing llama a `avanzarCarrera()`.
2. La vista llama al controlador.
3. El controlador llama al servicio.
4. El servicio llama a `Carrera.avanzar()`.

5. Carrera le pide a cada caballo que avance.
6. Cada caballo usa su estrategia de avance.
7. La vista recibe un CarreraEstadoDTO y repinta la pista.

La formula no busca ser realista, sino consistente y orientada a objetos. Tiene en cuenta:

- velocidad base;
- energia actual;
- resistencia;
- tipo de estrategia;
- variacion aleatoria;
- factor de rendimiento al iniciar la carrera.

6. Tipos de caballo

Velocista

Tiene una ventaja al inicio de la carrera.

Idea:

Sale mas fuerte al principio, pero no necesariamente mantiene esa ventaja hasta el final.

Clase:

- AvanceVelocista

Resistente

Tiene mejor rendimiento en la parte final.

Idea:

No siempre empieza ganando, pero puede recuperar terreno al final.

Clase:

- AvanceResistente

Equilibrado

No tiene un pico tan marcado, pero mantiene un rendimiento estable.

Idea:

Es regular durante toda la carrera.

Clase:

- AvanceEquilibrado

7. Por que no gana siempre el mismo caballo

Al principio habia un problema de balance: un caballo resistente podia ganar casi siempre porque su formula le daba demasiada ventaja.

Para corregirlo se agrego:

- una variacion aleatoria en cada avance;
- un factorRendimiento al iniciar cada carrera;
- formulas mas balanceadas entre velocistas, resistentes y equilibrados.

Esto significa que los atributos siguen importando, pero el resultado no queda fijo.

Frase para defenderlo:

Los caballos no ganan todos con la misma probabilidad porque sus atributos influyen. Pero tampoco gana siempre el mismo, porque incorporamos variacion de rendimiento y estrategias balanceadas. Eso hace que la simulacion sea simple, consistente y no deterministica.

8. MVC en el proyecto

MVC significa Modelo, Vista, Controlador.

Modelo

Paquete:

- model

Contiene la logica del dominio:

- jugador;
- caballo;
- carrera;
- pista;
- estrategias de avance.

Vista

Paquete:

- views

Contiene Swing:

- campos de texto;
- botones;
- combo de caballos;
- pista dibujada;
- mensajes.

Controlador

Paquete:

- controllers

Clase:

- JuegoController

Coordina entre vista y servicio.

Frase para defenderlo:

La vista no habla directamente con la base de datos ni manipula entidades del dominio. Se comunica con el controlador usando DTOs. Eso mantiene separadas las responsabilidades.

9. DTOs

Los DTOs son objetos simples para transportar informacion.

En el proyecto hay:

- CaballoDTO;
- JugadorDTO;
- CarreraEstadoDTO.

Ejemplo:

La vista no necesita recibir un objeto Carrera completo. Solo necesita saber:

- donde esta cada caballo;
- si termino la carrera;
- quien gano;
- cuantos puntos obtuvo el jugador;
- cual es el puntaje acumulado.

Eso viaja en CarreraEstadoDTO.

Frase para defenderlo:

Usamos DTOs para desacoplar la interfaz grafica del modelo interno. La vista recibe solo los datos que necesita mostrar.

10. Singleton

El proyecto usa Singleton en dos lugares importantes.

JuegoService

JuegoService mantiene el estado actual del juego:

- jugador actual;
- carrera actual;
- repositorios;
- servicio de puntaje.

Como la aplicacion de escritorio tiene una sola partida activa, se usa una unica instancia.

DatabaseConnection

Centraliza la conexion JDBC a MySQL.

Tambien crea las tablas si no existen:

- caballos;
- jugadores;
- carreras.

Frase para defenderlo:

Usamos Singleton cuando necesitamos una unica instancia compartida. En este caso, el servicio centraliza el estado de la partida y la clase de conexion centraliza el acceso a la base.

11. DAO y persistencia

DAO significa Data Access Object.

Sirve para separar la logica de negocio del acceso a datos.

En vez de escribir SQL dentro de JuegoService, el servicio usa interfaces:

- CaballoRepository;
- JugadorRepository;

- CarreraRepository.

Las clases MySQL implementan esas interfaces:

- MySQLCaballoRepository;
- MySQLJugadorRepository;
- MySQLCarreraRepository.

Esto permite cambiar la forma de persistir sin afectar tanto al negocio.

Ejemplo:

Si algun dia se cambia MySQL por PostgreSQL, se podria crear otra implementacion, por ejemplo PostgresJugadorRepository, sin cambiar la idea general del servicio.

Frase para defenderlo:

El DAO encapsula el SQL. El servicio pide guardar o consultar, pero no conoce detalles de tablas, columnas ni PreparedStatement.

12. Base de datos

La base se levanta con Docker usando MySQL.

Tablas principales:

caballos

Guarda los caballos disponibles.

Campos importantes:

- id;
- nombre;
- velocidad base;
- resistencia;
- tipo.

jugadores

Guarda jugadores creados desde la app.

Campos importantes:

- id;
- nombre;

- email;
- puntaje acumulado;
- caballo seleccionado.

carreras

Guarda resultados de carreras.

Campos importantes:

- jugador;
- caballo elegido;
- ganador;
- posicion del jugador;
- puntos obtenidos;
- fecha.

13. Swing y animacion

La interfaz grafica esta hecha con Swing.

Elementos usados:

- JFrame;
- JPanel;
- JLabel;
- JTextField;
- JComboBox;
- JButton;
- JOptionPane;
- Timer;
- Graphics2D.

La animacion funciona asi:

1. El usuario presiona "Iniciar carrera".
2. Se crea un Timer.
3. Cada tick del timer avanza la carrera.
4. Se actualiza el DTO de estado.
5. El panel de pista se repinta.
6. Cuando la carrera termina, se detiene el timer.

Frase para defenderlo:

Usamos Timer de Swing porque permite actualizar la interfaz de forma periodica sin hacer una simulacion compleja. Cada tick representa un instante de la carrera.

14. GRASP aplicado

Controller

JuegoController recibe los eventos del sistema desde la vista.

Ejemplo:

- crear jugador;
- iniciar carrera;
- avanzar carrera.

Information Expert

Carrera calcula el ganador y la posicion porque tiene la informacion necesaria:

- lista de caballos;
- distancia recorrida;
- pista.

Low Coupling

La vista no depende de SQL ni de entidades internas.

El servicio depende de interfaces de repositorio.

High Cohesion

Cada clase intenta tener una responsabilidad clara:

- PuntajeService calcula puntos;
- Caballo avanza;
- Carrera organiza la competencia;
- MainView muestra la pantalla;
- los DAOs persisten datos.

15. SOLID aplicado

Single Responsibility Principle

Cada clase tiene una razon principal para cambiar.

Ejemplos:

- Si cambia la formula de puntaje, se modifica PuntajeService.
- Si cambia la UI, se modifica MainView.
- Si cambia la base, se modifican los DAOs.

Open/Closed Principle

El sistema queda abierto a extension y cerrado a modificacion en la parte de estrategias.

Si se agrega un nuevo tipo de caballo, se puede crear una nueva clase que implemente EstrategiaAvance.

Liskov Substitution Principle

Todas las estrategias pueden usarse donde se espera una EstrategiaAvance.

Caballo no necesita saber si la estrategia concreta es velocista, resistente o equilibrada.

Dependency Inversion Principle

JuegoService trabaja con interfaces de repositorio, no directamente con clases concretas como contrato principal.

16. Recorrido tecnico de una carrera

Este es el recorrido mas importante para defender:

1. MainView detecta que el usuario presiono "Iniciar carrera".
2. MainView llama a JuegoController.iniciarCarrera().
3. JuegoController llama a JuegoService.iniciarCarrera(500).
4. JuegoService busca los caballos en CaballoRepository.
5. JuegoService crea una Carrera con el jugador, una Pista y los caballos.
6. MainView crea un Timer.
7. En cada tick, la vista llama a JuegoController.avanzarCarrera().
8. El controller llama a JuegoService.avanzarCarrera().
9. El service llama a Carrera.avanzar().
10. Carrera llama a Caballo.avanzar() para cada caballo.
11. Cada caballo usa su EstrategiaAvance.
12. Si un caballo llega a la meta, la carrera finaliza.
13. JuegoService calcula puntos con PuntajeService.

14. Se actualiza el jugador.
15. Se guarda el resultado en MySQL.
16. La vista muestra ganador, posicion y puntaje.

17. Preguntas que pueden hacer en la defensa

Por que usaron MVC?

Para separar responsabilidades. La vista se ocupa de mostrar, el controlador coordina acciones y el modelo contiene la logica del dominio.

Por que usaron DTOs?

Para no exponer entidades completas a la vista. La vista recibe solo la informacion necesaria para mostrar.

Donde esta la logica de negocio?

Principalmente en `model` y `services`. `Carrera` sabe avanzar y calcular ganador. `JuegoService` coordina el caso de uso completo. `PuntajeService` calcula los puntos.

Donde esta la persistencia?

En `repository.mysql` y `persistence`. Los DAOs usan JDBC y `DatabaseConnection` centraliza la conexion.

Que pasa si cambia la base de datos?

Se podria crear otra implementacion de las interfaces de repositorio. El servicio no deberia cambiar demasiado porque depende de interfaces.

Que pasa si agregamos otro tipo de caballo?

Se crea una nueva clase que implemente `EstrategiaAvance` y se actualiza la `factory` para construirla. No hace falta reescribir toda la clase `Caballo`.

Por que no gana siempre el mismo?

Porque el avance combina atributos, energia, estrategia, variacion aleatoria y factor de rendimiento por carrera.

Donde se guarda el puntaje?

El puntaje acumulado esta en `Jugador` y se actualiza en la tabla `jugadores`.

Que se guarda cuando termina una carrera?

Se guarda el jugador, el caballo elegido, el ganador, la posicion del jugador, los puntos obtenidos y la fecha.

18. Frases utiles para explicar en oral

- "La vista no contiene reglas de negocio; solo muestra datos y dispara acciones."
- "El controlador funciona como intermediario entre Swing y el servicio."
- "El modelo representa los conceptos del dominio: jugador, caballo, pista y carrera."
- "Carrera es Information Expert porque tiene la informacion necesaria para calcular ganador y ranking."
- "Usamos estrategias de avance para aplicar polimorfismo y evitar condicionales grandes."
- "Usamos DAO para que el SQL no quede mezclado con la logica de negocio."
- "Usamos DTOs para transportar datos entre capas sin exponer todo el modelo."
- "El resultado no es completamente deterministico porque se agrega variacion aleatoria controlada."
- "La persistencia esta separada, por eso cambiar la base o la forma de guardar no deberia impactar en la interfaz."

19. Como correr el proyecto

Desde la carpeta del proyecto:

```
./start.sh
```

El script:

1. levanta MySQL con Docker;
2. espera a que la base este lista;
3. descarga el conector de MySQL si hace falta;
4. compila el proyecto;
5. ejecuta la aplicacion Swing.

20. Resumen corto para decir de memoria

El sistema es una aplicacion Java Swing que simula una carrera de caballos. El usuario crea un jugador, elige un caballo e inicia la carrera. La interfaz esta separada del negocio usando MVC. La vista se comunica con el controlador mediante DTOs. El servicio coordina la creacion del jugador, el inicio de carrera, el avance y el guardado de resultados. El dominio esta modelado con clases como Jugador, Caballo, Carrera y Pista. Cada caballo avanza usando una estrategia distinta, aplicando polimorfismo. La base de datos MySQL se accede mediante DAOs con JDBC y una conexion Singleton. Al terminar la carrera, se calcula el puntaje segun la posicion del jugador y se guarda el resultado.